

Cornell Bowers C-IS

College of Computing and Information Science

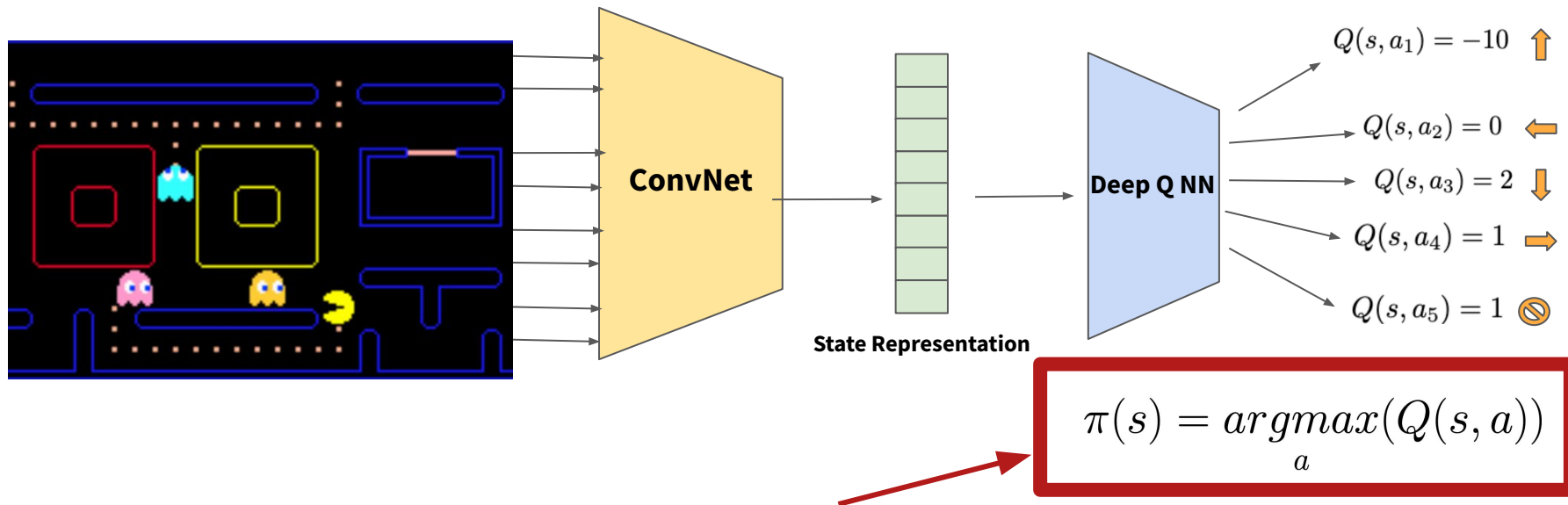
Deep Learning

Week 9: Policy Gradients

Thanks to

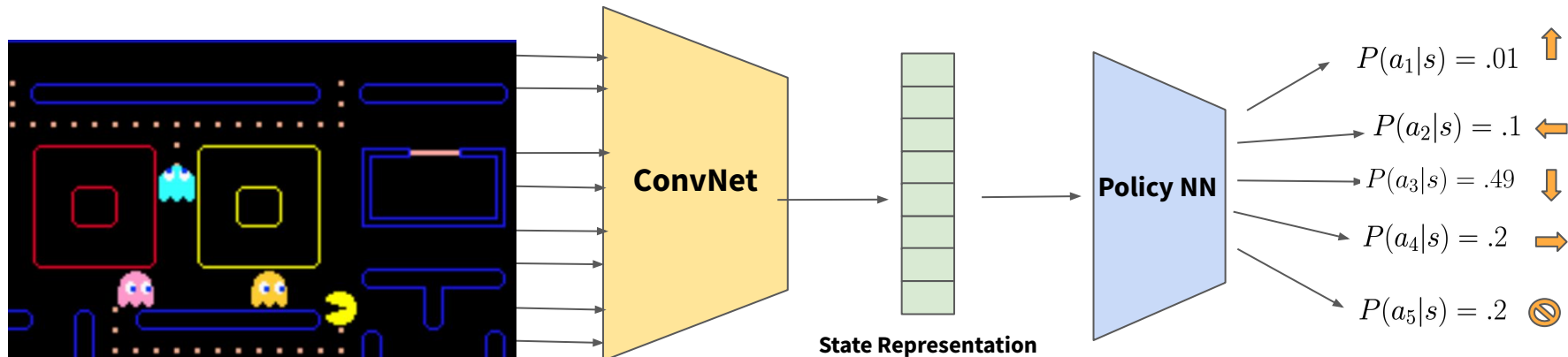
Varsha Kishore
Justin Lovelace
Luke Kulm
Jinzhou Li

Deep Q-Learning:



What if we can learn the policy directly?

Policy Gradient:

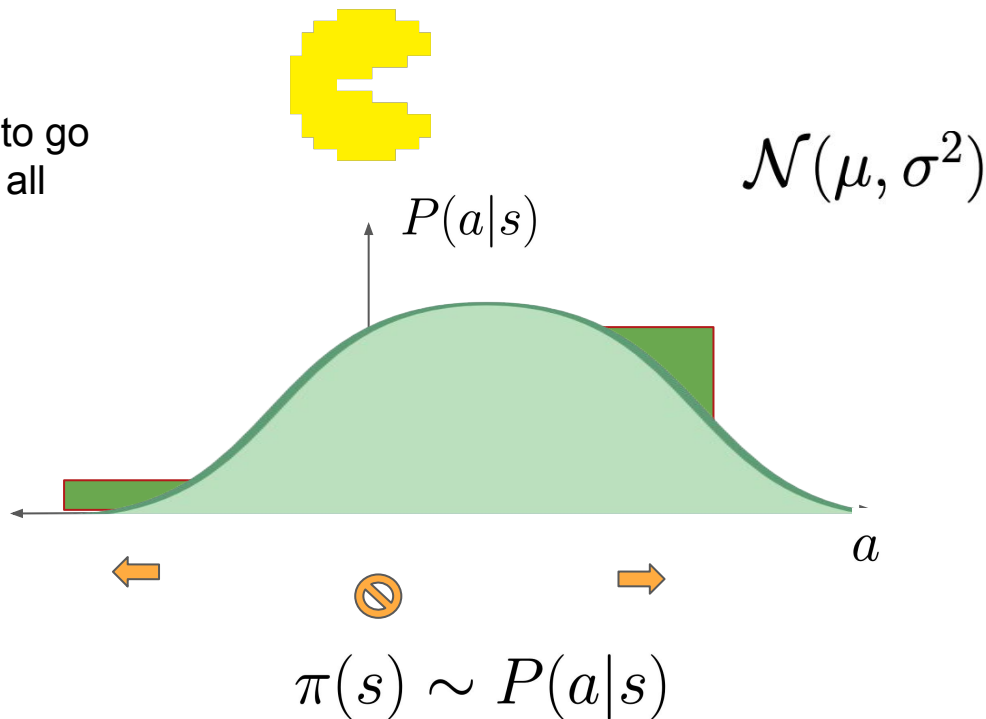


$$\pi(s) \sim P(a|s)$$

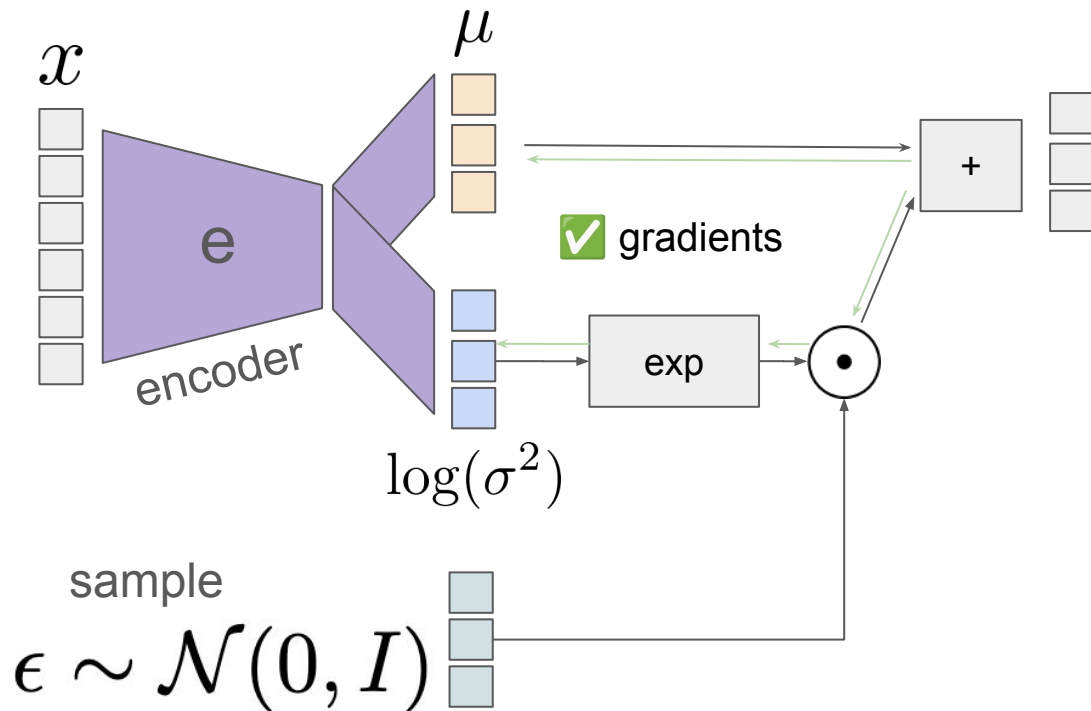
Policy Distribution:

Pacman is trying to decide to go left, right, or not move at all

Pacman is now trying to decide how fast to move?



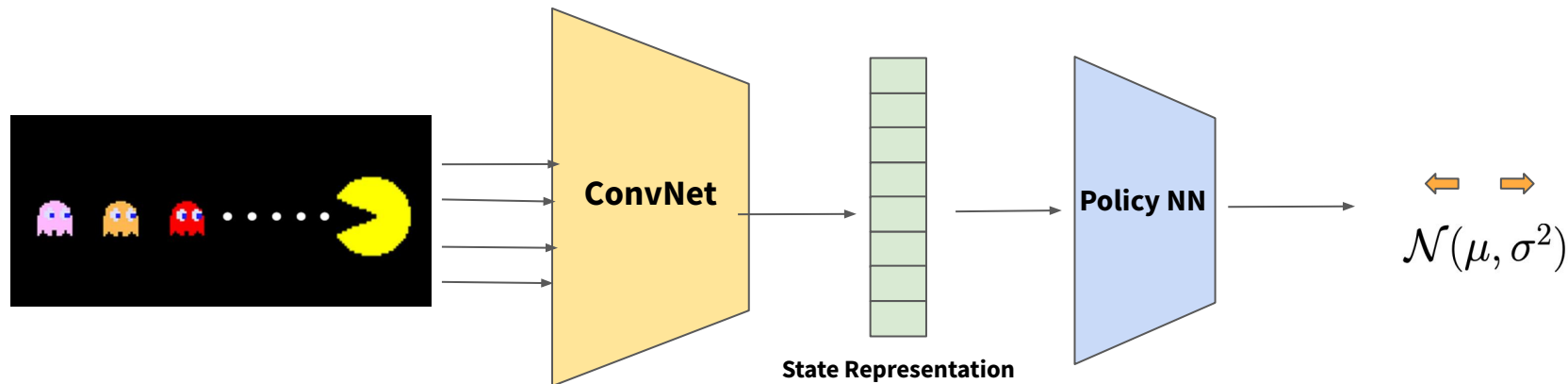
Review: The Reparameterization Trick



$$z = \mu + \sigma \odot \epsilon$$

$$z \sim \mathcal{N}(\mu, \sigma^2 I)$$

Continuous Policy Gradient:



$$\pi(s) \sim \mathcal{N}(\mu, \sigma^2)$$

Maximizing expected discounted reward

Let us denote one rollout of our agent following π_θ as:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$$

and define the discounted accumulated reward as

$$R(\tau) = \sum_{t=0}^T \gamma^t r_t$$

Given just the policy, the expected discounted reward is

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

How do we maximize expected reward? Gradient **ascent**!!

Gradient Descent:

$$\min_{\theta} \mathcal{L}(\theta)$$

update:

$$\theta_{k+1} = \theta_k - \alpha \nabla_{\theta} \mathcal{L}(\theta_k)$$

Policy Iteration:

$$\max_{\theta} J(\theta)$$

update:

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\theta_k)$$

$$\nabla_\theta J(\pi_\theta) = \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)]$$

$$= \nabla_\theta \sum_{\tau} P(\tau|\theta) R(\tau) \quad \text{Expand expectation}$$

$$= \sum_{\tau} \nabla_\theta P(\tau|\theta) R(\tau) \quad \text{Bring gradient under sum}$$

$$= \sum_{\tau} P(\tau|\theta) \nabla_\theta \log P(\tau|\theta) R(\tau) \quad \text{Log-derivative trick} \quad \nabla_\theta \log(z) = \frac{1}{z} \nabla_\theta z$$

$$= \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log P(\tau|\theta) R(\tau)] \quad \text{Return to expectation form}$$

$$\therefore \nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau) \right] \quad \text{Expression for grad-log-prob}$$

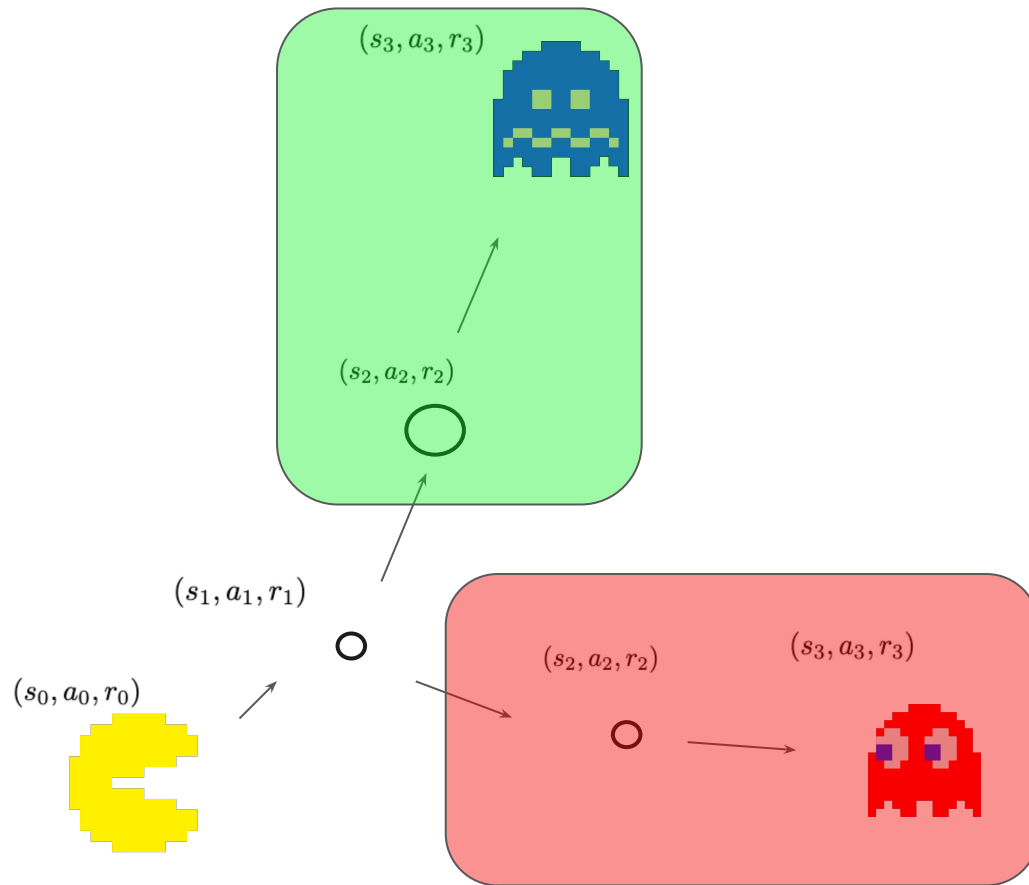
REINFORCE Algorithm

Estimate gradient with a single rollout

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

REINFORCE Algorithm:

1. Run a policy over the environment
2. Record actions, states, and rewards
3. Increase probability of good actions and decrease probability of bad actions



Let bygones be bygone

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\underset{\text{past}}{R(\tau_{k < t})} + \underset{\text{present+future}}{R(\tau_{k \geq t})} \right) \right]$$

REINFORCE Algorithm

Algorithm 1 REINFORCE: Monte-Carlo Policy-Gradient Control (episodic)
for π_*

Input: a differentiable policy parameterization $\pi(a|s, \theta)$

Algorithm parameter: step size $\alpha > 0$

Initialize policy parameter $\theta \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

repeat

 Generate an episode $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$, following $\pi(\cdot|\cdot, \theta)$

for $t = 0, 1, \dots, T - 1$ **do**

$$G_t \leftarrow \gamma^t \sum_{k=t+1}^T \gamma^{k-t-1} r_k$$

$$\theta \leftarrow \theta + \alpha G_t \nabla \ln \pi(a_t | s_t, \theta)$$

end for

until forever

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

REINFORCE Algorithm: Advantages and Disadvantages

Advantages:

- Unbiased estimate of the policy gradient.
- Can be applied to both continuous and discrete action spaces.

Limitations:

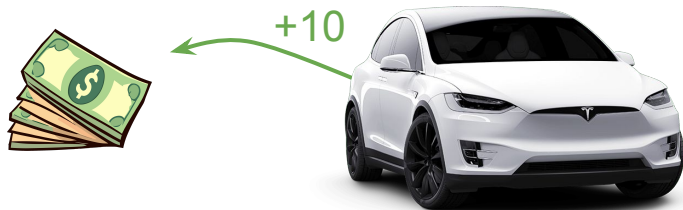
- Return is a noisy estimate of the action-value function.
 - High variance in the gradient estimates.
- High variance leads to potential instabilities and slow convergence.

Issues with Policy Gradient Methods

- Exploration
- High variance
- Reward hacking

Exploration

1. Might get stuck in Local Optima
2. Want to discover better policies
3. Adapt to changing environments
4. Generalize and handle uncertainty



Solutions to exploration?

1. Stochastic policy
2. Change our reward to emphasize exploration
 - a. Entropy bonus
3. Run algo from different start states



+1000

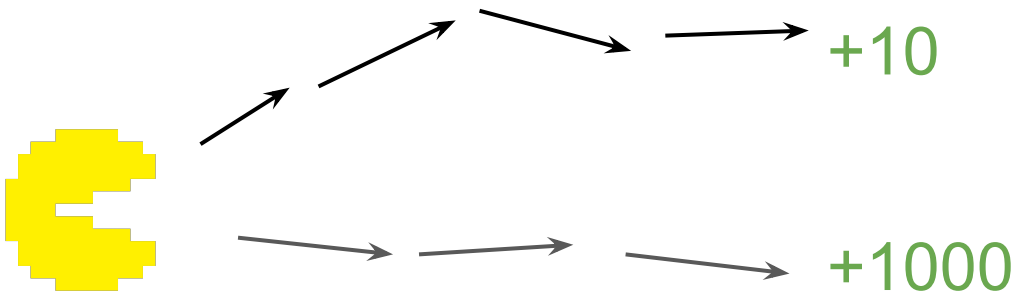


Variance

- Your policy is dependent on randomness
- Noise = high variance in how you evaluate your policy
- Need relative information about how good a policy is

Is this a good
sequence of actions?

$$\hat{g} = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t$$

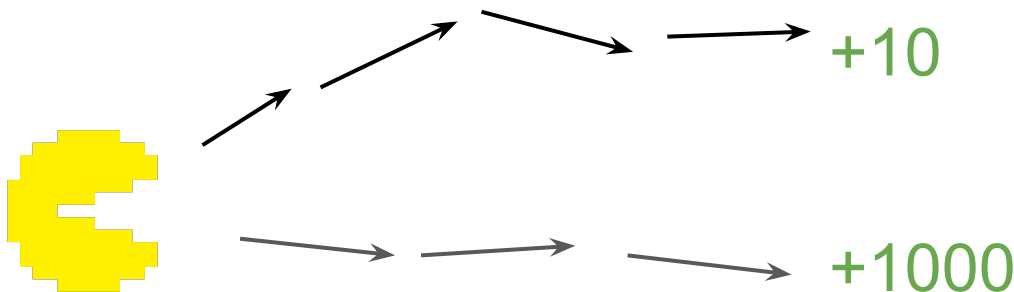


Variance Reduction with a Baseline

How good was that sequence of actions compared to other sequences?

$$\hat{g} = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t))$$

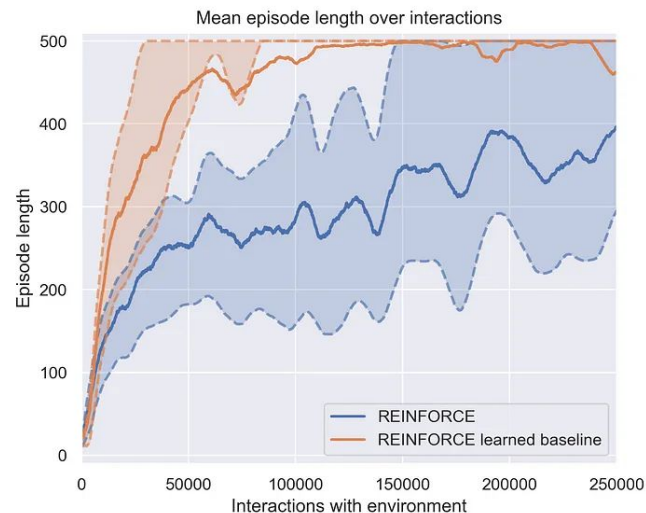
Subtract a Baseline



Variance Reduction with a Baseline

- Subtracting a baseline is a simple technique to reduce variance in the REINFORCE algorithm.
 - Improves learning stability and convergence speed.
- A baseline, $b(s_t)$, estimates the expected return from state s_t .
 - Can be an estimated state-value function $V^{\pi_{\theta}}(s_t)$ or a moving average of returns.
- The policy gradient estimate becomes:

$$\hat{g} = \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t))$$



Actor-Critic Algorithms

Combine the benefits of both policy-based and value-based approaches.

Key Components:

- **Actor:** The actor is a policy network $\pi_{\theta}(a|s)$ that maps states to probability distributions over actions. It selects actions based on the current policy.
- **Critic:** The critic is a value network $V_{\phi}(s)$ or $Q_{\phi}(s, a)$ that estimates the expected return or action-value function. It evaluates the quality of the actor's decisions.

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(a|s) Q_{\phi}(s, a)$$

Actor-Critic Algorithms

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi_{\theta}(a|s) Q_{\phi}(s, a)$$

Moving right sounds
like a good choice



Don't do that. Bad
Idea!



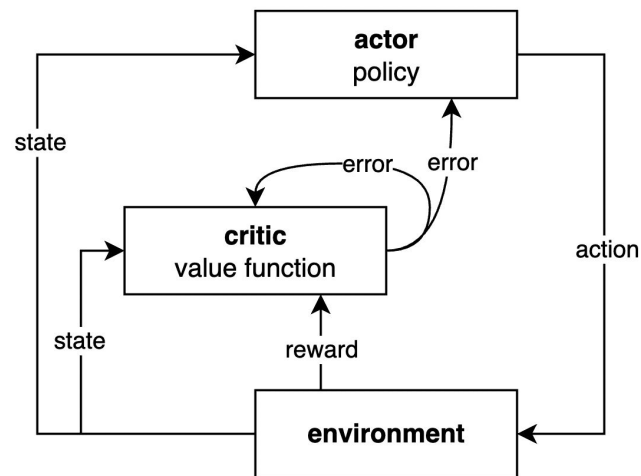
Actor-Critic Algorithm

Algorithm 3 Actor-Critic Algorithm (Q-Function Critic)

```
1: Initialize actor network  $\pi_\theta(a|s)$  with random weights  $\theta$ 
2: Initialize critic network  $Q_\phi(s, a)$  with random weights  $\phi$ 
3: for each episode do
4:   Initialize state  $s$ 
5:   for each step of the episode do
6:     Choose action  $a \sim \pi_\theta(a|s)$ 
7:     Take action  $a$ , observe reward  $r$  and next state  $s'$ 
8:     Choose next action  $a' \sim \pi_\theta(a|s')$ 
9:     Compute TD error:  $\delta = r + \gamma Q_\phi(s', a') - Q_\phi(s, a)$ 
10:    Update critic weights  $\phi$  using TD learning:
11:       $\phi \leftarrow \phi + \alpha_c \delta \nabla_\phi Q_\phi(s, a)$ 
12:    Compute policy gradient:
13:       $\nabla_\theta J(\theta) = \nabla_\theta \log \pi_\theta(a|s) Q_\phi(s, a)$ 
14:    Update actor weights  $\theta$  using policy gradient ascent:
15:       $\theta \leftarrow \theta + \alpha_a \nabla_\theta J(\theta)$ 
16:     $s \leftarrow s'$ 
17:   end for
18: end for
```

Actor-Critic: Advantages

- Combine the **benefits** of **policy**-based and **value**-based methods.
- The critic helps in **reducing the variance** of the policy gradient estimates.
- The actor allows for **continuous** and **stochastic** action spaces.
- Can be more **sample-efficient** compared to pure policy-based methods.



Challenges with RL in the Real World

Sample Inefficiency + Danger + Cost = Simulation



Reward hacking

- Learn to maximize the reward in unexpected ways.
- How do we get what we want, NOT what we say we want?
- It is important to have a good reward function

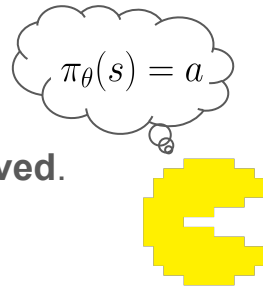


On-policy vs. Off-policy

Reinforcement learning algorithms can be categorized as either on-policy or off-policy based on how they collect and use data for learning.

On-Policy Algorithms:

- Learn from experiences generated by the **current policy** being **followed** and **improved**.
- The **policy** used for **generating** data is the **same** as the policy being **improved**.

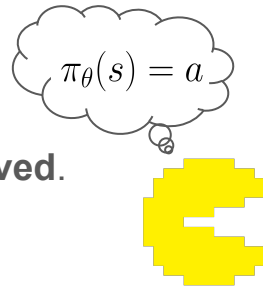


On-policy vs. Off-policy

Reinforcement learning algorithms can be categorized as either on-policy or off-policy based on how they collect and use data for learning.

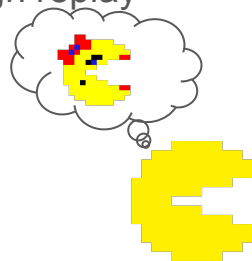
On-Policy Algorithms:

- Learn from experiences generated by the **current policy** being **followed** and **improved**.
- The **policy** used for **generating** data is the **same** as the policy being **improved**.



Off-Policy Algorithms:

- Learn from experiences generated by a **different** policy than the one being **improved**.
- The behavior policy (used for **generating** data) is **different** from the target policy (being **learned**).
- Can learn from data generated by **any policy** and enables efficient use of data through replay buffers.



Discuss: Which of the following are on- vs. off-policy algorithms?

Q-Learning, Deep Q-Learning, REINFORCE

On-Policy Learning

Advantages:

- Simpler to implement and understand
- More stable learning process with fewer hyperparameters to tune
- Better performance in certain tasks requiring specialized behaviors
- Convergence properties are often better understood theoretically
- Can be more suitable for problems where exploration must be carefully controlled

Disadvantages:

- Less sample efficient as data can only be used once
- Requires new data collection for every policy update
- Exploration is more challenging since it must be built into the policy being learned
- Cannot learn from demonstrations or historical data
- Usually requires more environment interactions to achieve good performance

Off-Policy Learning

Advantages:

- More sample efficient through experience replay
- Can learn from demonstrations, historical data, or other agents
- Better exploration-exploitation balance through separate behavior policies
- Can reuse past experiences multiple times for learning
- Enables learning optimal policies while following exploratory behavior

Disadvantages:

- Often more complex to implement correctly
- Can suffer from instability issues during learning
- May require importance sampling corrections for policy differences
- Typically needs more hyperparameter tuning
- Can be prone to overestimation bias in value functions

Recap

- Policy gradient methods directly learn the policy function
 - Policy Gradient Theorem lets us estimate the gradient of expected reward with respect to the policy parameters
- REINFORCE learns the policy network parameters with an unbiased estimate of the gradient
 - But can have high variance!
 - Exploration must be integrated somehow
- Off-policy vs. On-policy algorithms both have their place with pros and cons
- Actor-Critic algorithms introduce a learnable critic to estimate the gradient
 - Reduces variance
- It is important to have a good reward function (that cannot easily be hacked)